EV Charging Network Backend

AI-Assisted Machine Coding Problem | Time limit: 2 hours

We need to build a backend for an electric-vehicle charging network. A driver opens the app, finds a station near them with a free connector, plugs in, charges, unplugs, and gets billed for the energy delivered.

You are expected to use AI coding tools (Claude, Cursor, Copilot, ChatGPT, etc.) for this exercise — in fact, we encourage it. Because AI removes most of the boilerplate effort, the bar is higher than a traditional machine coding round: we are evaluating your design judgment, your ability to direct and review AI-generated code, and how deeply you understand and own everything you submit.

Concretely: **working code is the price of entry, not the achievement.** We are more interested in the decisions you made — what you chose not to build, where you put the seams, and what you decided when the spec was silent — than in how much of the scope you covered.

Mandatory Functional Requirements

- Register a driver, along with their vehicle.
- Register a charging station at a location, with one or more connectors.
- Start a charging session if a compatible connector is free at a station within a certain radius of the driver.
- End a session, given the energy delivered, and get the cost for it.
- Take a connector out of service and bring it back into service. A connector that is out of service must not be offered to anyone.
- Implement a tiered tariff. One example, for a DC connector: minimum session charge of ₹150; first 10 kWh at ₹20/kWh; 11–25 kWh at ₹14/kWh; 26 kWh and beyond at ₹9/kWh.
- Show session history for a driver: active and completed sessions.
- Show session history for a station: active and completed sessions.
- Support different connector types — **AC** (slow) and **DC** (fast) — with different tariffs for each. If a driver requests an AC connector and none is free, start them on a DC connector billed at the AC tariff.
- Support promo codes: add a promo code, delete a promo code, and apply a valid promo code when starting a session to get a discounted price.

The tariff example above is illustrative and not fully specified. Where the requirements are ambiguous, decide, and tell us what you decided. We are more interested in your reasoning than in a particular answer.

Bonus Requirements (attempt only after the mandatory scope is solid)

- **Peak-hour tariff** — a pluggable multiplier based on time of day or how loaded a station is.
- **Configurable station-selection strategy** — e.g., nearest station vs. cheapest vs. highest-power — switchable without touching session logic.
- **Session cancellation** with a no-show fee policy.
- **Concurrency safety** — handle the race condition where two drivers claim the last free connector at the same time.

Deliverables (Non-Negotiable)

1. **Working code** — CLI app or API-based web app, your choice. In-memory storage is fine, but the domain modeling must be complete. Any language, framework, or open-source library is allowed.
2. **Automated tests** — at minimum covering the billing engine: tariff slabs, minimum session charge, connector-type tariffs, promo code application, and the AC-billed-on-DC case.
3. **A short README** covering: your assumptions; key design decisions and trade-offs; what you would do differently with more time; and a brief note on how you used AI — what you prompted for, what you rejected or rewrote, and why.
4. **The repository, including its .git directory.** Send us the repo itself or push to a private repo and add us — we want to see how the solution evolved, not just where it landed. Please do not send a zip of the working directory alone.

Ground Rules for AI Use

- You may use any AI tool freely — there are no restrictions on prompting.
- **You own every line.** "The AI wrote it" is never an acceptable answer; if you can't explain a piece of code, remove it or rewrite it.
- Commit incrementally with meaningful messages so we can see how the solution evolved.
- Blindly accepted AI output with dead code, inconsistent abstractions, or untested logic will count against you more than a smaller, well-understood solution.

Demo (30 Minutes)

Time	Activity
8 min	Show the working solution in action, including at least one edge case (e.g., no station in radius, a session below the minimum charge, an AC request served on a DC connector, an invalid promo code).
8 min	Walk through your design and code: layering, where each strategy lives, and why. **We will choose which files we look at.**
10 min	**Live extension exercise:** we will ask you to make a small change on the spot (for example, add a battery-swap connector type, a new tariff slab, or a flat-amount promo code). You may use AI during this too. We are watching how you scope the change, where you make it, and how you verify it.
4 min	Q&A. Expect deep questions about any part of the code, including parts AI generated.

What We Evaluate

- **Ownership** — whether you can explain, defend, and change every line under questions you didn't choose. Not what percentage the AI wrote; a solution that is 95% generated and 100% understood is a strong submission.
- **Judgment** — what you did when the specification was silent or ambiguous, and whether you wrote that decision down.
- **Design and extensibility** — where the seams are, and whether a new tariff rule, connector type, or selection strategy lands in one place or five.
- **Workable solution** — correctness of matching, billing, connector-type handling, and promo codes. Shortcuts are fine if you can justify them and describe the better solution.
- **Verification** — the quality of your tests and how you convinced yourself the code is correct.

Points to Note

- Feel free to assume any missing detail — call out all assumptions at the start of the demo.
- Using a database is optional; in-memory storage is acceptable.
- You do not need to simulate charging over time. Ending a session takes the energy delivered as an input.
- Please reach out if you are stuck anywhere or need help. We want this to be an enjoyable experience for you.